

NodeJS e nossa primeira API

Parte 2 — Continuando a construção da nossa API

Sumário

- Revisão

- Nossa primeira rota

- CORS

- O que é
 - Como usar

- Continuando nossa implementação

- Buscar todos
 - Buscar um
 - Criar
 - Funções JS que vamos usar
 - Editar (PUT e PATCH)
 - Deletar

Revisão

Nossa primeira rota

Até aqui, já construímos os 3 pilares de uma API com Express

```
import express from "express";

const app = express(); //primeiro pilar: instancia do express

app.get("/", (req, res) => {
  res.send("seja bem vindo à gestao de livros");
});

app.get("/livros", function (pergunta, resposta) {
  //segundo pilar: rotas
  resposta.send("Hello world!");
});

app.listen(3000); //terceiro pilar: porta a ser ouvida
```

- Instância do Express
- Rotas (app.get)
- Porta a ser ouvida (app.listen)

CORS

CORS — O que é

- Cross-Origin Resource Sharing
- Mecanismo de segurança do próprio navegador
 - Por padrão, o navegador bloqueia requisições feitas para um domínio/porta diferente do site atual
- Exemplo do problema:
 - Seu front roda em localhost:5173
 - Sua API roda em localhost:3000
 - O navegador barra a requisição antes mesmo dela chegar na API
- Precisamos autorizar, explicitamente, quem pode acessar nossa API

CORS — Como usar

Instalamos o pacote cors

Registramos ele como um middleware, com app.use



```
npm install cors
```



```
import cors from "cors";
```

```
const app = express();
```

```
app.use(cors()); // libera o acesso para qualquer origem
```

Continuando nossa implementação

Buscar todos

Devolve a lista inteira de usuários cadastrados em memória



```
const usuarios = [  
  { id: 1, nome: "João", idade: 25 },  
  { id: 2, nome: "Maria", idade: 30 },  
];
```

```
app.get("/usuarios", (req, res) => {  
  res.json(usuarios);  
});
```

Buscar um

O :id na rota é um parâmetro dinâmico

req.params.id sempre chega como texto, por isso convertemos com parseInt

```
app.get("/usuarios/:id", (req, res) => {  
  const id = parseInt(req.params.id, 10);  
  const usuario = usuarios.find((u) => u.id === id);  
  
  if (!usuario) {  
    return res.status(404).json({ erro: "Usuário não encontrado" });  
  }  
  
  res.json(usuario);  
});
```

Criar

Precisamos habilitar o `express.json()` para conseguir ler o body da requisição



```
app.use(express.json()); // middleware que lê o body em JSON
```

```
app.post("/usuarios", (req, res) => {
```

```
  const novoUsuario = {
```

```
    id: usuarios.length + 1,
```

```
    nome: req.body.nome,
```

```
    idade: req.body.idade,
```

```
  };
```

```
  usuarios.push(novoUsuario);
```

```
  res.status(201).json(novoUsuario);
```

```
});
```

Funções básicas do JS

(Parse, Typeof, Find, FindIndex, Slice, Splice)

Para editar e deletar, vamos precisar de...

- Parse (converter)
- ParseInt (converter para inteiro)
- Typeof (verificar o tipo)
- Find (encontrar)
- FindIndex (encontrar a posição)
- Slice (cortar, sem alterar o original)
- Splice (cortar/alterar o original)

Parse (numéricos)

Como o nome sugere, esse tipo de função serve para converter valores de um tipo para outro

parseInt

parseFloat



```
let meu_valor = "10" //isso e um valor do tipo STRING, texto
```

```
//se eu quiser que seja um numero, preciso converter
```

```
let meu_valor_numerico = parseInt(meu_valor, 10);
```

Parse (string)

Usado para converter valores (qualquer valor) em texto



```
let meu_numero = 10;  
let usuario_ativo = true;  
  
let meu_numero_em_texto = String(meu_numero); //"10"  
let usuario_ativo_em_texto = String(usuario_ativo); //"true"
```

Typeof (verificar o tipo de um dado)

Função que retorna qual o tipo do dado, podendo ser:

booleano (verdadeiro ou falso) · inteiro · ponto flutuante · string · objeto · função



```
console.log(typeof 123);           // "number"
console.log(typeof 3.14);          // "number"
console.log(typeof "Olá");          // "string"
console.log(typeof true);           // "boolean"
console.log(typeof undefined);      // "undefined"
console.log(typeof null);           // "object"
console.log(typeof [1, 2, 3]);       // "object"
console.log(typeof { nome: "Ana" }); // "object"
console.log(typeof function () {}); // "function"
```

Find

Percorre um array e devolve o primeiro elemento que atende uma condição

Passa de elemento em elemento, procurando satisfazer a condição

Se encontrar o que procura, para de procurar



```
for (let index = 0; index < usuarios.length; index++) {  
  const usuario = usuarios[index];  
  if (/*minha condicao*/) {  
    console.log(usuario);  
    break; // para nao continuar procurando  
  }  
}
```

Find

Problema: encontre o primeiro usuário que tenha mais de 30 anos



```
const usuarios = [  
  { nome: "João", idade: 25 },  
  { nome: "Maria", idade: 10 },  
  { nome: "Cleber", idade: 52 },  
  { nome: "Matheus", idade: 6 },  
  { nome: "Suzana", idade: 22 },  
  { nome: "Camila", idade: 25 },  
  { nome: "Jonathan", idade: 24 },  
  { nome: "Fernando", idade: 34 },  
  { nome: "Clara", idade: 41 },  
  { nome: "Bruno", idade: 18 },  
];  
  
const maiorDeTrinta = usuarios.find((u) => u.idade > 30);
```

FindIndex

Parecido com o find, porém, o find devolve um registro, enquanto o findIndex devolve a posição desse registro

FIND

"AQUI ESTÁ O REGISTRO
QUE VOCÊ PEDIU"

FINDINDEX

"OS DADOS QUE VOCÊ QUER
ESTÃO NA POSIÇÃO X DO ARRAY"

FindIndex

Problema: encontre a posição do primeiro usuário que tenha mais de 30 anos



```
const usuarios = [  
  { nome: "João", idade: 25 },  
  { nome: "Maria", idade: 10 },  
  { nome: "Cleber", idade: 52 },  
  { nome: "Matheus", idade: 6 },  
  { nome: "Suzana", idade: 22 },  
  { nome: "Camila", idade: 25 },  
  { nome: "Jonathan", idade: 24 },  
  { nome: "Fernando", idade: 34 },  
  { nome: "Clara", idade: 41 },  
  { nome: "Bruno", idade: 18 },  
];  
  
const posicao = usuarios.findIndex((u) => u.idade > 30);
```

E por que eu precisaria saber a posição de alguma coisa?

[1, 2, 3, 4]

aqui está o registro que você precisa deletar

Slice (cortar)

“Corta” uma parte do array, e devolve um novo
ele não altera o array original

```
const frutas = [  
  "banana", "maçã", "laranja", "uva", "abacaxi",  
  "kiwi", "morango", "cereja", "pera", "melancia",  
];  
  
const apenas_as_primeiras_5_frutas = frutas.slice(0, 5);
```

Splice

Serve para adicionar, remover ou substituir itens dentro de um array, a partir de uma POSIÇÃO específica diferente do slice, ele altera o array original



```
const frutas = ["kiwi", "morango", "cereja", "pera", "melancia"];
```

```
frutas.splice(1, 1); // remove "morango"
```

```
console.log(frutas);
```

Editar — PUT

PUT substitui o recurso inteiro pelos dados enviados

```
● ● ●  
app.put("/usuarios/:id", (req, res) => {  
  const id = parseInt(req.params.id, 10);  
  const index = usuarios.findIndex((u) => u.id === id);  
  
  if (index === -1) {  
    return res.status(404).json({ erro: "Usuário não encontrado" });  
  }  
  
  usuarios[index] = { id, nome: req.body.nome, idade: req.body.idade };  
  
  res.json(usuarios[index]);  
});
```

Editar — PATCH

PATCH atualiza só os campos enviados, mantendo o restante do recurso

```
● ● ●  
app.patch("/usuarios/:id", (req, res) => {  
  const id = parseInt(req.params.id, 10);  
  const index = usuarios.findIndex((u) => u.id === id);  
  
  if (index === -1) {  
    return res.status(404).json({ erro: "Usuário não encontrado" });  
  }  
  
  usuarios[index] = { ...usuarios[index], ...req.body };  
  
  res.json(usuarios[index]);  
});
```

Deletar

Usamos `findIndex` para achar a posição e `splice` para remover

```
app.delete("/usuarios/:id", (req, res) => {  
  const id = parseInt(req.params.id, 10);  
  const index = usuarios.findIndex((u) => u.id === id);  
  
  if (index === -1) {  
    return res.status(404).json({ erro: "Usuário não encontrado" });  
  }  
  
  usuarios.splice(index, 1);  
  
  res.status(204).send();  
});
```

Organizando nosso projeto

Estrutura básica de um projeto Node

Conhecer a estrutura básica de projetos Node

Organizar nosso código de maneira elegante



Controllers e Routes

routes define os endpoints e encaminha para o controller

controller concentra a lógica de cada operação

```
● ● ●  
  
// routes/users-router.js  
import { Router } from "express";  
import * as UserController from "../controller/user-controller.js";  
  
const router = Router();  
  
router.get("/", UserController.buscarTodos);  
router.get("/:id", UserController.buscarUm);  
router.post("/", UserController.criar);  
router.put("/:id", UserController.editar);  
router.patch("/:id", UserController.editarParcial);  
router.delete("/:id", UserController.deletar);  
  
export default router;
```

app.use e Middlewares

Middleware: função que roda entre a requisição e a resposta

Recebe (req, res, next) e precisa chamar next() para o fluxo continuar

Exemplos: express.json(), cors(), um logger customizado



```
function logger(req, res, next) {  
  console.log(`${req.method} ${req.url}`);  
  next(); // sem isso, a requisicao trava aqui  
}
```

```
const app = express();
```

```
app.use(logger);
```

```
app.use(cors());
```

```
app.use(express.json());
```

```
app.use("/usuarios", usersRouter);
```

Obrigado!

Próxima aula: persistindo os dados em um banco de verdade